CHAPTER 9

The Design of Active Record

Active Record is the library in Rails that maps Ruby objects to database records. This functionality is called *object-relational mapping*, and it allows you to get the best of both the relational database (used for persistence) and object-oriented programming (used for business logic).

In this chapter, as well as the next two, we'll take a look at the high-level design of Active Record's source code and how its pieces fit together. We are less interested in *what* Active Record does than *how* it does it. All we need is a very short example of mapping a class to a database—just enough to kick-start our exploration of Active Record's internals.

A Short Active Record Example

Assume that you already have a file-based SQLite database that follows Active Record's conventions: this database contains a table called `ducks`, which has a field called `name`. You want to map the records in the `ducks` table to objects of class `Duck` in your code.

Let's start by requiring Active Record and opening a connection to the database. (If you want to run this code on your system, you also need to install the SQLite database and the `sqlite3` gem. But you can probably follow along fine by just reading the example, without running it.)

part2/ar_example.rb

```
require 'active_record'
ActiveRecord::Base.establish_connection :adapter => "sqlite3",
                                        :database => "dbfile"
```

Note that in a Rails application, you don't need to worry about opening the connection; the application reads the names of the adapter and the database from a configuration file, and it calls `establish_connection` for you. We're using Active Record on its own here, so we have to open the connection ourselves.